Setting Up the Two-Tier System on a New Mac

Everything from a blank Mac to a working system, including training the small language models

This manual takes a Mac that has nothing installed on it and ends with the complete system running: the small local models, the learned router, the cache, Claude as the escalation model and examiner, the dashboard, and your own fine-tuned versions of the two small language models (SLMs). Every step says what to type, and what you should see, so you know it worked.

It is written for macOS with **zsh** (the default Terminal since macOS Catalina), on both **Apple Silicon** (M1, M2, M3, M4) and **Intel** Macs. The general manual (`Two-Tier-System-User-Manual.md`) covers Windows as well and explains the system in more depth; this one is the Mac route only, and adds the things that are specific to a Mac.

> **Read this first: you cannot train the models on the Mac itself.** The training scripts use Unsloth with 4-bit training, which needs an **NVIDIA graphics card**, and no current Mac has one. So the project trains the SLMs on **Google Colab** (a free-to-rent NVIDIA computer in your browser), controlled from your Mac, and then brings the finished models back to the Mac to run them. Section 9 does that. Everything else, including *running* the trained models (Ollama uses the Mac's own GPU), happens on your Mac. Nothing in this manual was run on a physical Mac by the author: the commands are standard macOS ones and the project's own code and package list were checked on Linux, so so keep section 14 (problems and fixes) handy.

What is in this manual

0. What runs where, what it costs, how long it takes
1. Prepare the Mac
2. Install Python, Git and Ollama
3. Download the project
4. Create the Python environment
5. Give the project your Claude key
6. Put models in Ollama so you can try the system now
7. Check everything and run for the first time
8. Open the dashboard
9. Train the two SLMs on Google Colab, from your Mac
10. Bring the trained models onto the Mac and into Ollama
11. Retrain the router for the new models
12. Check the finished system
13. Daily use: the commands you will actually type
14. Mac problems and fixes
15. The whole setup as a checklist

---

0. What runs where, what it costs, how long it takes

Job	Where it happens	Why
Running the pipeline, the router, the cache, the dashboard	Your Mac	Ordinary Python; no special hardware
Answering with the small models	Your Mac, in **Ollama**	Ollama uses the Mac's GPU automatically (Apple Silicon: Metal)
Claude (escalations, quality marking)	Anthropic's computers, through your key	A paid service
Training the SLMs	**Google Colab** (NVIDIA GPU), from your browser	The training code needs an NVIDIA graphics card and CUDA
Training the router	Your Mac	A small text model; takes seconds

The two routes for the small models. You do not have to train anything to get a working system:

- **Route A: base models** (sections 6 to 8). Download two ready-made models. About 30 minutes. Fully working, but the answers and the router's behaviour differ from the fine-tuned models the project was measured with.
- **Route B: fine-tuned models** (sections 9 to 11). Train on Colab, then import. About 3.7 and an estimated 5.5 hours of waiting on Colab (payments measured, retail estimated, on its L4 GPU), plus an hour of work. This is the full system.

The sensible order is to do Route A first (so you know everything works on your Mac), then Route B.

What you need

Item	Details
A Mac	Any recent Mac. **8 GB of memory is the least that works** (16 GB is comfortable); about **20 GB of free disk** (packages about 5 GB, models 4 to 8 GB, plus working room). Apple Silicon is faster; Intel works
macOS	A recent version. Ollama sets its own minimum: check ollama.com/download
Internet	For downloads, Claude and Colab
Accounts	**Anthropic** (a Claude API key, paid pay-as-you-go), **GitHub** (optional), **Google** (for Colab training only)

Time. Sections 1 to 8: one to two hours, mostly downloads. Training on Colab: about 3.7 hours (payments) plus 5.5 hours (retail bank), mostly waiting.

Money. The local models are free. Claude: a 100-question run with every option on costs roughly **$0.15 to $0.25**, almost all of it quality marking (about $0.0012 per mark); `--noquality` makes it a few cents. Colab: its paid tiers charge "compute units" per hour; the author's L4 used about 1.5 units an hour, so both trainings took about 14 units. These are estimates from the project's logs; prices change, so read them from Anthropic's and Google's own pages.

---

1. Prepare the Mac

1.1 Open Terminal

Press `Cmd + Space`, type `Terminal`, press Enter. A window opens with a line ending in `%`. **This is where you type every command in this manual.** Nothing you type runs until you press Enter. To paste, use `Cmd + V`.

1.2 Install the Apple command line tools

These give your Mac the basic developer programs (including Git). Type:

```
xcode-select --install
```

A window asks you to install; click **Install** and wait (a few minutes). If it says "already installed", that is fine. **Check:** `xcode-select -p` prints a path such as `/Library/Developer/CommandLineTools`.

1.3 Find out which kind of Mac you have

```
uname -m
```

It prints `arm64` (**Apple Silicon**) or `x86_64` (**Intel**). Remember which: a few paths below differ.

1.4 Install Homebrew

Homebrew is the standard tool for installing programs on a Mac, and it keeps the next steps short.

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

It explains what it will do and asks for your Mac password (you will not see characters as you type it; that is normal). When it finishes, it prints "Next steps". **Do them**, which for a Mac come down to this:

- **Apple Silicon (`arm64`):**

  ```
  echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
  eval "$(/opt/homebrew/bin/brew shellenv)"
  ```

- **Intel (`x86_64`):**

  ```
  echo 'eval "$(/usr/local/bin/brew shellenv)"' >> ~/.zprofile
  eval "$(/usr/local/bin/brew shellenv)"
  ```

Check: `brew --version` prints a version. If it says "command not found", you skipped the two lines above.

1.5 Choose where the project will live

Make a folder for it **outside** iCloud-synced folders:

```
mkdir -p ~/Projects
cd ~/Projects
```

Avoid putting the project in Desktop or Documents if "iCloud Drive: Desktop & Documents Folders" is switched on in your settings: iCloud syncing the project's log files while the system writes to them can cause delays and conflicts, and macOS may keep asking for permission to access those folders.

---

2. Install Python, Git and Ollama

2.1 Python 3.12 and Git

```
brew install python@3.12 git
```

Check:

```
python3.12 --version
git --version
```

You should see `Python 3.12.x` and a Git version. **Use Python 3.12 rather than the `python3` that comes with the Mac:** Apple's own copy (`/usr/bin/python3`) is often older than the 3.11 this project needs. (The project was also tested on 3.14, but 3.12 is the safe choice for the machine-learning packages on a Mac.)

Alternative: you can install Python from python.org/downloads instead. If you do, afterwards run `open "/Applications/Python 3.12/Install Certificates.command"` once, or downloads will fail with `CERTIFICATE_VERIFY_FAILED`. The Homebrew route does not have this problem.

2.2 Ollama

Ollama runs the small language models on your Mac.

1. Go to **ollama.com/download**, click **Download for macOS**, and open the downloaded file.
2. Drag **Ollama** into your **Applications** folder, then open it from there. If macOS says it cannot verify the app, open **System Settings, Privacy & Security**, scroll down, and click **Open Anyway** next to Ollama's name.
3. A small llama icon appears in the menu bar at the top right. **Ollama must be running (that icon visible) whenever you use the system.** The first time, it may offer to install its command-line tool: click **Install**.

Check (in Terminal):

```
ollama --version
```

This project needs 0.12.11 or newer, because older versions do not return the word probabilities that Tier 1's confidence is computed from. If you see "command not found", open the Ollama app once, then open a new Terminal tab (`Cmd + T`) and try again.

2.3 Node.js (optional)

Only if you want to change how the dashboard looks. `brew install node`. You do not need it to use the dashboard.

---

3. Download the project

```
cd ~/Projects
git clone https://github.com/bipul-ranjan/two-tier-system.git
cd two-tier-system
```

You should see lines scrolling by, ending with `Resolving deltas: 100% done`. **Stay in this folder (`~/Projects/two-tier-system`) for every command from here on.** Check you are in the right place: `ls` should list `src`, `scripts`, `data`, `training`,

`dashboard`, `README.md` and more.

For your own copy of the repository, or to save changes back to GitHub, you need a **personal access token** instead of a password: on github.com, click your picture, **Settings, Developer settings, Personal access tokens, Tokens (classic), Generate new token (classic)**, tick **repo**, generate, and copy the `ghp_...` code at once (it is shown once). Use your GitHub username when Git asks for a username and the token when it asks for a password. macOS remembers it in the Keychain.

---

4. Create the Python environment

This makes a private box for the project's software so it never clashes with anything else on your Mac. (It also avoids the `externally-managed-environment` error that Homebrew's Python gives if you try to install packages outside an environment.)

```
python3.12 -m venv venv
source venv/bin/activate
```

You should see `(venv)` at the start of the line. From now on, **every time you open a new Terminal window or tab to work on the project, do these two things first:**

```
cd ~/Projects/two-tier-system
source venv/bin/activate
```

Inside the environment `python` works (you no longer need `python3.12`). **Check:** `python --version` shows 3.12, and on an Apple Silicon Mac `python -c "import platform; print(platform.machine())"` prints `arm64`. If it prints `x86_64` on Apple Silicon, your Terminal is running under Rosetta and everything will be slow; open a normal Terminal (Finder, Applications, Utilities).

Now install the packages:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
pip install -r dashboard/backend/requirements.txt
```

The first is large (several GB, because the cache and the router's research tools use PyTorch), so it takes several minutes and prints a lot. That is normal.

Check:

```
python -m pytest tests/ -q
```

You should see `45 passed`. These tests need no models and no key, so they confirm the Python side is healthy. Then check the scikit-learn version, which matters because the router's saved model only loads under the version that trained it:

```
python -c "import json; print('router needs:', json.load(open('models/router/text_probe_router_info.json'))
['sklearn_version'])"
python -c "import sklearn; print('you have:   ', sklearn.__version__)"
```

If the two numbers match (at the time of writing a fresh install gives 1.9.1, which is what the router was trained with), you are fine. If they differ, either `pip install "scikit-learn==<the first number>"` or retrain the router in section 11. The pipeline checks this itself and tells you.

---

5. Give the project your Claude key

Escalated questions and quality marking use Claude, so the project needs your own private key.

1. Go to **console.anthropic.com** (not claude.ai; the "console" is for developers). Sign up or log in.
2. **Settings, Billing**: add a payment card. It is pay-as-you-go; normal use costs cents.
3. **API Keys, Create Key**, name it `two-tier-system`, create it, and **copy the key at once** (it starts `sk-ant-` and is never shown again).
4. Save it for the project **without leaving it in your command history**. Type this, press Enter, paste the key when it

asks (nothing appears as you paste; that is normal), and press Enter:

```
read -s "KEY?Paste your Claude key, then press Enter: "
echo "export ANTHROPIC_API_KEY=\"$KEY\"" >> ~/.zshrc
unset KEY
source ~/.zshrc
```

5. **Check** that it is set, without printing the whole secret:

   ```
   echo ${ANTHROPIC_API_KEY:0:7}
   ```

   It should print `sk-ant-`. If it prints nothing, repeat step 4. A **new** Terminal window reads `~/.zshrc` automatically.

Keep the key private. Never paste it into a chat, an email, or a file in the project, and never commit it to GitHub: anyone who has it can spend money on your account. If you think it has leaked, delete it in the console and make a new one.

---

6. Put models in Ollama so you can try the system now

(This is **Route A**. Section 9 onward replaces these with fine-tuned models.)

The project's settings name the **fine-tuned** models, which do not exist on your Mac yet. For now, point it at two ready-made models. Make sure Ollama is running (menu-bar icon), then download them, one at a time, waiting for each to finish (each is a few gigabytes):

```
ollama pull phi3:mini
ollama pull qwen2.5:1.5b
```

Check: `ollama list` shows both. Now tell the project to use them. This one line changes only the two model names in `src/config.py`:

```
sed -i '' 's/"payment-assistant-v2"/"phi3:mini"/; s/"retail-bank-assistant-v2"/"qwen2.5:1.5b"/' src/config.py
```

(On a Mac, `sed -i` needs the `''` after it. Do not remove it.) **Check:** `python -m src.config` prints each business unit with the model it will use: `phi3:mini` and `qwen2.5:1.5b`. You can undo this edit at any time with `git checkout src/config.py`, which you will do in section 10.

Talk to a model directly to prove Ollama works:

```
ollama run phi3:mini "Why was my card payment declined?"
```

You should see an answer. Type `/bye` to leave.

---

7. Check everything and run for the first time

7.1 Check each part

```
python -m src.tier1
```

One real answer from the payments model, with its confidence. The first call is slow while Ollama loads the model.

```
python -m src.tier2_escalate
```

One real Claude answer. This proves your key works and costs a fraction of a cent.

7.2 The router: use it, or run without it

The project includes a trained router in `models/router/`, but it was trained on the **fine-tuned** models' answers, not the base models'. For Route A, it still works, but its predictions are less well matched. For a first run, either is fine. Just try the default:

```
python -m src.pipeline 10
```

What you should see: a line `Router: LEARNED (default)`, the models loading, one block of text per question (the answer, its confidence, `Router : ... predicted quality 3.44 (cut-point 3.27)`, and a `Decision` of `LOCAL`, `CACHE` or `ESCALATE`), then a stage of **quality scoring** with progress counts, and a summary. Ten questions take a few minutes.

- If it stops at the start with a message about the router, follow the message: it means the scikit-learn version differs (section 4) or the model file is missing. To run once without the learned router: `python -m src.pipeline 10 --threshold-router`.
- The first run also downloads a small embedding model (about 90 MB) for the cache.

Check the results were saved: `ls results/logs` shows `results_history.csv` and `results_log_combined.csv`.

7.3 The options

```
python -m src.pipeline [N] [--threshold-router] [--nocache] [--noquality] [--no-redact] [--score-quality-local MODEL]
```

Option	Effect
`N`	How many random practice questions (default 100)
`--threshold-router`	Use the older rule (escalate when confidence is below a pass mark) instead of the learned router
`--nocache`	Do not re-use old Claude answers
`--noquality`	Skip the Claude quality marking (cheaper). Fill it in later with `python scripts/backfill_quality_scores.py`
`--no-redact`	Turn the personal-data filter off. It is on by default and replaces card numbers, emails, phones and similar identifiers before text goes to Claude or the log. Use only to reproduce older runs
`--score-quality-local MODEL`	Also mark with a local Ollama model into separate columns (a cheap extra opinion)

Keep the Mac awake during long runs (below 100 questions it rarely matters): run `caffeinate -dimsu python -m src.pipeline 300` and the Mac will not sleep until it finishes.

---

8. Open the dashboard

```
python -m uvicorn dashboard.backend.main:app --port 8000
```

You should see lines ending with `Uvicorn running on http://127.0.0.1:8000`. **Leave that Terminal window open.** Open Safari or Chrome and go to **http://localhost:8000**. You will see the run you just made: how many questions stayed local, were served from the cache, or went to Claude; the router and its cut-point; quality; times; cost; and trend charts. The page refreshes every five seconds, so you can watch a run happen. To stop the dashboard, click in its Terminal window and press `Ctrl + C`.

If the port is busy, use `--port 8010` and open that address.

You now have a working system on your Mac (Route A). The rest of this manual trains the small models and upgrades the system to the full version.

---

9. Train the two SLMs on Google Colab, from your Mac

9.1 What training does

The two base models (Phi-3-mini and Qwen2.5-1.5B) are taught to be better at banking questions by showing them thousands of example questions and good answers. Only a small set of extra adjustable weights is trained (about 1.5% of the model for Phi-3-mini and 2.3% for Qwen), and the result is exported as a single `.gguf` file that Ollama can load. Payments trains on 10,902 examples; the retail-bank model on 18,843 (14,643 real plus 4,200 synthetic ones about fraud, hardship and similar situations).

9.2 Before you start

- A **Google account**, and **Chrome** is the safest browser for Colab (Safari works for most things but is more often a

source of odd problems).

- Expect **about 3 hours 44 minutes** for payments and **about 5.5 hours** for the retail-bank model on Colab's **L4** GPU (the free T4 is about twice as slow). Colab's free tier may not give you a GPU for that long, or at all at busy times; a paid tier (Colab Pro, which charges "compute units") is the dependable way. Read the current rate on Colab's own screen.
- **Keep the Mac awake and the Colab tab open for the whole run.** Closing the lid puts a Mac to sleep and Colab disconnects. In a **separate** Terminal tab, run:

  ```
  caffeinate -dimsu
  ```

  and leave it running; it stops the Mac sleeping until you press `Ctrl + C`. Keep the Mac plugged in. (If a session still drops, you do not lose the work: see 9.6.)

Everything in this section is typed into Colab in your browser, not into Terminal. Colab works in "cells": click **+ Code**, paste, and press the **Play** button (or `Shift + Enter`).

9.3 Open Colab with a GPU, and connect Google Drive

1. Go to **colab.research.google.com**, sign in, click **New notebook**.
2. **Runtime, Change runtime type**, choose **L4 GPU** (or T4 if that is what you have), **Save**.
3. Connect Drive. This is where your trained models and checkpoints are saved, because Colab's own storage is wiped when a session ends or when you change the GPU type:

   ```
   from google.colab import drive
   drive.mount('/content/drive')
   ```

   A pop-up asks you to sign in and allow access. **You should see** `Mounted at /content/drive`.

9.4 Download the project into Colab (onto your Drive)

```
%cd /content/drive/MyDrive
!git clone https://github.com/bipul-ranjan/two-tier-system.git
%cd /content/drive/MyDrive/two-tier-system
```

(If that folder already exists from an earlier session, run `!git pull` inside it instead of cloning.) If you use a **private** copy of the repository, store a GitHub token as a Colab Secret (the key icon in the left sidebar, name it `GITHUB_TOKEN`, switch "Notebook access" on) and clone with `token = userdata.get('GITHUB_TOKEN')` rather than pasting the token into a cell, because Colab saves cells with their output.

Always run every Colab command from this project folder. The training scripts look for `data/raw/...` relative to it.

9.5 Install, fetch the data, and train the Payment Assistant

```
!pip install -r training/requirements.txt
```

A few minutes, with a lot of scrolling text. Then the training data:

```
!python scripts/fetch_bitext_banking.py
```

You should see it download the dataset and write `data/raw/bitext_payments...` and `bitext_retail_bank...` files. Now the long step:

```
!python training/train_payments.py
```

A stream of numbers scrolls by (the "loss" falls as the model learns): that is normal. It saves a **checkpoint to Drive every 250 steps** and, at the end, exports the finished model to `LJMU_Research/two-tier-system-models` on your Google Drive. **This takes about 3 hours 44 minutes on an L4.** The line it prints when done tells you where the model went.

9.6 If the session drops

Colab sessions can disconnect (the tab closed, the Mac slept, or a time limit). Nothing is lost: the checkpoints are on Drive.

Open a **new** notebook, repeat 9.3 and 9.4 (mount Drive, `git pull`), run `!pip install -r training/requirements.txt` again, and **run the same training command**. The script finds the last checkpoint and prints "Found an existing checkpoint ... resuming from there instead of starting over".

9.7 Restart, then train the Retail Bank Assistant

1. **Runtime, Restart runtime.** This clears the first model from the GPU's memory; skipping it can cause an out-of-memory error. A restart wipes the whole session, so repeat 9.3 (mount Drive) and 9.4 (`%cd` into the project folder, `!git pull`), and `!pip install -r training/requirements.txt` and `!python scripts/fetch_bitext_banking.py` again if they were lost.
2. Make the retail model's extra training data and train it:

   ```
   !python scripts/generate_exception_training_data.py
   !python training/train_retail.py
   ```

3. **Read the first lines the exception-data script prints:** "Loaded N existing pipeline-test instructions to avoid duplicating". **N must be well above 0** (about 16,000). That step builds training examples for fraud and hardship scenarios and removes any that also appear among the project's test questions, so the models are never tested on what they trained on. If N is 0, **stop**: the protection is not working (usually because you are not in the project folder).
4. The retail training takes **about 5.5 hours** on an L4, and saves its model to the same Drive folder.

9.8 Check what you have on Drive

In Colab you can look with `!ls -R /content/drive/MyDrive/LJMU_Research/two-tier-system-models | head -30`. You are looking for a `.gguf` file for each model, with names based on `payment_assistant` and `retail_bank_assistant` (Unsloth may place each in a folder such as `payment_assistant_gguf`, perhaps with a `Modelfile`). There will also be `payments_checkpoints` and `retail_checkpoints` folders: those are only for resuming and can be deleted later.

Training is done. You do not need Colab again unless you retrain. Press `Ctrl + C` in the `caffeinate` Terminal tab so the Mac can sleep normally again.

---

10. Bring the trained models onto the Mac and into Ollama

10.1 Get the files onto your Mac

The simplest way is **Google Drive for desktop**:

1. Go to **google.com/drive/download**, download **Drive for desktop** for Mac, install it, and sign in with the same Google account you used for Colab.
2. Click its menu-bar icon, then the gear, **Preferences**, **Google Drive**, and choose **Mirror files** (a real copy on your Mac) rather than "Stream files". Wait for the syncing icon to finish. The model files are a couple of gigabytes each, so this can take a while. If you used Stream, right-click the files in Finder and choose **Available offline**.
3. Find the models. Recent macOS keeps Google Drive at `~/Library/CloudStorage/GoogleDrive-<your email>/My Drive/`. Let the Mac search for you:

   ```
   find ~/Library/CloudStorage -name "*.gguf" 2>/dev/null
   ```

   It prints the full path of each `.gguf` file. Note the folder holding the payments one and the one holding the retail-bank one.

(Instead of Drive for desktop, you can download each file from drive.google.com in the browser; large files are slower and may ask you to confirm a virus-scan warning.)

10.2 Create the two Ollama models

Do the payments model first. Go to the folder that holds its `.gguf` file (use the path `find` printed; put the path in quotes because it contains spaces):

```
cd "/Users/YOUR-NAME/Library/CloudStorage/GoogleDrive-YOU@gmail.com/My Drive/LJMU_Research/two-tier-system-models/payment_assistant_gguf"
ls
```

Create the one-line file Ollama needs (it names your `.gguf` automatically; this only works if the folder holds **one** `.gguf`, which `ls` will show):

```
echo "FROM ./$(ls *.gguf | head -n 1)" > Modelfile
cat Modelfile
```

`cat` shows one line such as `FROM ./payment_assistant.gguf` (the name will match your file). Now create the model, using **exactly** this name, because `src/config.py` expects it:

```
ollama create payment-assistant-v2 -f Modelfile
```

You should see it read the file and report `success`. Now the retail-bank model, in *its* folder:

```
cd "/Users/YOUR-NAME/Library/CloudStorage/GoogleDrive-YOU@gmail.com/My Drive/LJMU_Research/two-tier-system-models/retail_bank_assistant_gguf"
echo "FROM ./$(ls *.gguf | head -n 1)" > Modelfile
ollama create retail-bank-assistant-v2 -f Modelfile
```

(If Unsloth already put a `Modelfile` in the folder, you may use that one instead; if you overwrite it with the line above, nothing is lost for this project.)

Check:

```
ollama list
ollama run payment-assistant-v2 "Why was my card payment declined?"
ollama run retail-bank-assistant-v2 "How do I close my current account?"
```

Both names should be listed, and each should answer in the style of a bank assistant. Type `/bye` to leave each.

10.3 Point the project at them

Go back to the project, and undo the base-model edit from section 6 (this restores `payment-assistant-v2` and `retail-bank-assistant-v2`):

```
cd ~/Projects/two-tier-system
source venv/bin/activate
git checkout src/config.py
python -m src.config
python -m src.tier1
```

`python -m src.config` should list the two `-v2` model names, and `python -m src.tier1` should print a real answer from the fine-tuned payments model.

---

11. Retrain the router for the new models

The router learned the habits of the models whose answers were in its history. The one in the repository learned from the project's own fine-tuned models, so if yours trained the same way it should already fit. But your models are your own training run, so for the best match, build history with them and retrain. The router needs **at least 200 rows with Tier 1 quality marks over at least 5 question types** (more is better).

1. **Start the history afresh** if you ran anything with the base models in Route A, so the router does not learn from the wrong models' answers:

   ```
   mv results/logs results/logs-base-models
   ```

   (Nothing is deleted; the old logs are just set aside. If you never ran Route A, skip this.)

2. Make history with the fine-tuned models. The learned router is not trained for them yet, so use the older rule for this

one run; it costs about $0.15 per 100 questions, mostly for marking. 300 questions is the minimum worth doing, and 1,000 is better:

```
caffeinate -dimsu python -m src.pipeline 500 --threshold-router
```

(about 1 to 2 hours; Claude marks the answers at the end).

3. Train the router:

   ```
   python scripts/train_text_router.py
   ```

   You should see a table comparing "Tier 1 confidence" with "Learned router" (the router's correlation with Claude's marks should be clearly higher), the cut-point and the share it would escalate in each business unit, and finally `Saved models/router/text_probe_router.joblib`. It takes seconds. To escalate fewer questions: `python scripts/train_text_router.py --target-share 0.25`.

4. Have a look at what it learned: `open models/router/what_it_learned.txt`.

---

12. Check the finished system

```
python -m pytest tests/ -q                      # 45 passed
python -m src.pipeline 100                      # the full system: learned router, cache, Claude, marking
python -m uvicorn dashboard.backend.main:app --port 8000
```

On the dashboard, the latest run should show: its router named (a version such as `text-probe-...`) and a cut-point; the three outcomes; and quality marks for every answer. The run list tags the first run after the router changed with **new router**. To compare the two routers fairly, open `src/pipeline.py`, change `SEED = None` to `SEED = 42`, and run `python -m src.pipeline 100` and `python -m src.pipeline 100 --threshold-router`.

To keep your trained router in the project's repository, commit the new files in `models/router/` (`git add models/router`, `git commit -m "Retrained router"`, `git push`).

---

13. Daily use: the commands you will actually type

Every session starts the same way:

```
cd ~/Projects/two-tier-system
source venv/bin/activate
```

and Ollama must be running (the llama icon in the menu bar).

To do this	Type this
Run the system on 100 questions	`python -m src.pipeline 100`
...cheaper (no quality marking)	`python -m src.pipeline 100 --noquality`
...without the cache	`python -m src.pipeline 100 --nocache`
...with the older confidence router	`python -m src.pipeline 100 --threshold-router`
...and make sure every mark is filled in	`python scripts/run_and_score.py 100`
Keep the Mac awake for a long run	`caffeinate -dimsu python -m src.pipeline 500`
Fill in missing quality marks	`python scripts/backfill_quality_scores.py` then `python scripts/backfill_quality_scores.py --draft`
Retrain the router	`python scripts/train_text_router.py`
Open the dashboard	`python -m uvicorn dashboard.backend.main:app --port 8000`, then browse to `http://localhost:8000`
Summary tables for the latest run	`python -m src.evaluate`
Which models will it use?	`python -m src.config`
Which models does Ollama have / have loaded?	`ollama list` / `ollama ps`

Run the tests	`python -m pytest tests/ -q`
Rescue a run not saved to the history	`python scripts/recover_missing_run.py`
Get the latest project changes	`git pull`
Leave the environment	`deactivate`

The log files are in `results/logs/` (`results_history.csv` holds every run); open them in Numbers or Excel, but not while a run is in progress. `results/logs/README.md` explains all 47 columns.

---

14. Mac problems and fixes

What you see	What it means, and what to do
`zsh: command not found: brew`	The two "shellenv" lines from step 1.4 were skipped. Run them, then open a new Terminal tab
`zsh: command not found: ollama`	The Ollama app has not been opened yet, or its command-line tool was not installed. Open the app, click **Install** if asked, open a new Terminal tab
`zsh: command not found: python`	The environment is not active. Run `source venv/bin/activate` from the project folder. (Outside the environment use `python3.12`)
`error: externally-managed-environment` when running `pip`	You are installing outside the environment. Activate it first (section 4)
`No module named ...`	The environment is not active: look for `(venv)` at the start of the line
`CERTIFICATE_VERIFY_FAILED`	You installed Python from python.org. Run `open "/Applications/Python 3.12/Install Certificates.command"` once. (The Homebrew Python does not need this)
macOS says an app "cannot be opened because the developer cannot be verified"	**System Settings, Privacy & Security**, scroll down, click **Open Anyway**
The router message says the scikit-learn version differs	`pip install "scikit-learn==<version in the message>"`, or retrain (section 11)
`model "payment-assistant-v2" not found`	The fine-tuned model is not in Ollama. Do section 10, or point `src/config.py` at the base models (section 6)
`Connection refused` to Ollama, or answers never arrive	Ollama is not running. Open the app (llama icon in the menu bar)
Every answer takes 20 seconds or more, or a warning about models not "resident"	The two models do not both fit in memory, so they swap. Close other apps (browser tabs, Photos, Xcode), or check `ollama ps`
`Killed: 9` or the whole Mac slows to a crawl	Out of memory. Close other apps and run fewer questions at a time. 8 GB Macs are tight with a browser open
Your Claude questions fail with an authorization error	The key is not set in this window. Open a **new** Terminal tab (it reads `~/.zshrc`), activate the environment, and check `echo ${ANTHROPIC_API_KEY:0:7}`
`PermissionError` / "locked" on `results_history.csv`	Another run (or a program that has it open) is using it. Close it. If a run finished but is missing, `python scripts/recover_missing_run.py`
The Mac went to sleep and a run, or Colab, stopped	Use `caffeinate -dimsu` (sections 7 and 9). For Colab, a stopped run resumes from its checkpoint (9.6)
Files in the project seem to change or reappear on their own	The project is inside an iCloud-synced folder. Move it to `~/Projects` (section 1.5)
Python reports `x86_64` on an Apple Silicon Mac	Terminal is running under Rosetta. Use the normal Terminal app, not a Rosetta copy
`Address already in use` starting the dashboard	Another program uses port 8000. Add `--port 8010` and open that address
The dashboard shows an old layout or old numbers	Hard-refresh: `Cmd + Shift + R`
Colab: "Google Drive is not mounted"	Run the mount cell first (9.3)
Colab: `data/raw/... not found`	You are not in the project folder (`%cd /content/drive/MyDrive/two-tier-system`), or the data step was skipped
Colab: out of memory on the second model	Runtime, Restart runtime, then repeat the setup before `train_retail.py` (9.7)
Colab: the session stops partway	Resume (9.6): same command again, from the last checkpoint
`find ~/Library/CloudStorage -name "*.gguf"` prints nothing	Drive for desktop has not finished syncing, or is on **Stream**: switch to **Mirror files** or mark the files **Available offline**
`ollama create` says the file is not found	The `Modelfile` must be in the same folder as the `.gguf`, and `FROM ./<exact file name>` must match. Run `ls` and `cat Modelfile`

If you meet an error not listed, copy the exact message somewhere safe before doing anything else; it usually says what is

wrong. The general manual (section 19) and each folder's README also list problems.

One more honest limit. Training on a Mac itself (for example with Apple's MLX tools) is possible in principle, but it is a different method from the project's training scripts, which are written for Colab's NVIDIA GPUs. It is not set up, tested or described here, and models trained that way would need their own conversion to `.gguf` and their own measurements.

---

15. The whole setup as a checklist

Tick each when done. The "Check" is what proves it.

On the Mac (Route A, a working system)

- [ ] Terminal open; `xcode-select -p` prints a path
- [ ] `uname -m` noted (`arm64` or `x86_64`); `brew --version` works
- [ ] `~/Projects` created; project is **not** in an iCloud-synced folder
- [ ] `brew install python@3.12 git`; `python3.12 --version` shows 3.12
- [ ] Ollama app installed and running (menu-bar llama); `ollama --version` is 0.12.11 or newer
- [ ] `git clone ...` done; `ls` in `~/Projects/two-tier-system` shows `src`, `scripts`, `data`
- [ ] `python3.12 -m venv venv`; `source venv/bin/activate` shows `(venv)`
- [ ] `pip install -r requirements.txt` and `-r dashboard/backend/requirements.txt` finished
- [ ] `python -m pytest tests/ -q` shows **45 passed**
- [ ] scikit-learn version matches the router (or you will retrain)
- [ ] Claude key saved with `read -s`; `echo ${ANTHROPIC_API_KEY:0:7}` prints `sk-ant-`
- [ ] `ollama pull phi3:mini` and `ollama pull qwen2.5:1.5b`; `sed` edit applied; `python -m src.config` shows them
- [ ] `python -m src.tier1` and `python -m src.tier2_escalate` each print a real answer
- [ ] `python -m src.pipeline 10` finished; `results/logs/results_history.csv` exists
- [ ] Dashboard opens at `http://localhost:8000` and shows the run

Training the SLMs (Route B)

- [ ] Google account, Colab with an **L4** (or T4) GPU; `caffeinate -dimsu` running in a spare Terminal tab
- [ ] Drive mounted; project cloned into `MyDrive`; `%cd` into it
- [ ] `pip install -r training/requirements.txt`; `scripts/fetch_bitext_banking.py` ran
- [ ] `training/train_payments.py` finished (about 3 h 44 min); a `.gguf` is on Drive
- [ ] Runtime restarted; setup repeated; `generate_exception_training_data.py` reported **N well above 0**
- [ ] `training/train_retail.py` finished (about 5.5 h); a second `.gguf` is on Drive
- [ ] Drive for desktop on **Mirror files**; `find ~/Library/CloudStorage -name "*.gguf"` lists both
- [ ] `Modelfile` created for each; `ollama create payment-assistant-v2` and `ollama create retail-bank-assistant-v2` both said `success`
- [ ] `ollama list` shows both `-v2` models; both answer with `ollama run`
- [ ] `git checkout src/config.py`; `python -m src.config` shows the `-v2` names; `python -m src.tier1` answers

Finish

- [ ] Old logs set aside (`mv results/logs results/logs-base-models`) if you ran Route A
- [ ] `python -m src.pipeline 500 --threshold-router` finished (history built with the new models)
- [ ] `python scripts/train_text_router.py` saved a router
- [ ] `python -m src.pipeline 100` runs with the learned router; the dashboard shows it
- [ ] `python -m pytest tests/ -q` shows 45 passed